

Sri Lanka Institute of Information Technology



IT3012: Intelligent Agents - Lab 1

Y3.S1.WD.AI.01.02

Malabe Campus

Student ID -	IT24103099
Student Name -	Uthpala P.B.H
Module Name -	Software Engineering Frameworks
Module Number -	IT3012
LabSheet Number -	1
LabSheet Date -	2026/July/28

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" branch of the repo

Task 1.1

The Environment State (`__init__`) 1. (Remember) List the four components of the PEAS framework discussed in Lecture 01

- P - Performance Measure
- E – Environment
- A – Actuators
- S – Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

- `self.width,`
- `self.height,`
- `self.walls,`
- `self.food_positions,`
- `self.opponents,`
- `self.agent_pos`

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

- Classification: Multi-Agent Environment Reason: The environment includes `self.opponents`, which represents one or more opponent agents.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

- A percept sequence is the complete history of all the percepts an agent has received from the environment since it started operating.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

- Sensors (S)
- The `get_percept()` method gathers information from the environment

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

- Fully Observable. Because the `get_percept()` method provides the agent with all the relevant information about the current state, including its position, opponent positions, food, walls, score etc.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

- Performance Measure (P)

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

- Because the Performance Measure should evaluate the agent's success in the environment, not its internal processing. Hitting a wall is an external event that shows poor performance.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. Implementation Instructions: Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position (0, 0), walls, and food.

```
def get_percept(self, agent) -> dict:
    return {
        'agent_pos': list(self.agent_pos),
        'smells_food': tuple(self.agent_pos) in self.food_positions,
        'hit_wall': tuple(self.agent_pos) in self.walls,
        'score': self.score,
        'remaining_food': len(self.food_positions),
        'smells_toxin': tuple(self.agent_pos) in self.toxic_traps
    }
```

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

- Creates a new set called `self.toxic_traps`. Then randomly generates 3 trap positions. Ensures traps that are : not at the starting position (0,0), not on walls, not on food locations.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. Implementation Instructions: Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

```
# Check if agent stepped on a toxic trap
if tuple_pos in self.toxic_traps:
    self.score -= 15
```

10. (Analyze) By adding 'smells_toxin' , you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

- Checks if the agent is standing on a toxic trap. Then smells_toxin becomes true, Otherwise smells_toxin becomes false.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. Implementation Instructions: In execute_action , check if the updated self.agent_pos intersects with self.toxic_traps to decrement self.score by 15 points. In draw_grid , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

- The Vacuum World exploit showed that an agent can exploit a bad scoring system by repeatedly cleaning the same dirt to earn more points, instead of keeping the environment clean.